My Rust Journey — Presentation Script

Target runtime: 15–20 minutes

SLIDE 1 — Title: "My Rust Journey"

[Advance to slide, pause 3 seconds for audience to read]

Good [morning/afternoon], everyone.

Today I want to share the journey I've been on over the past several months learning Rust — one of the most challenging and rewarding programming languages in the world.

We'll cover where I started, the structured curriculum I followed, the core concepts I had to master, and then I'll walk you through **six real projects** I built along the way — from a simple guessing game all the way up to fully-deployed web applications and a multi-crate game engine.

This will run about fifteen to twenty minutes, and I'll leave time for questions at the end. All of the code I mention today is publicly available on GitHub — links are on the final slide.

Let's dive in.

SLIDE 2 — Agenda

[Read through the agenda briefly]

Here's the roadmap for today. We'll start with *why* Rust — what makes it different from other languages. Then I'll describe the structured study plan I followed. After that we get into the good stuff: core concepts and then each project in turn. We'll close with key takeaways and what comes next.

SLIDE 3 — Why Rust?

[Pause on each row for a moment]

So why Rust? There are hundreds of programming languages — why this one?

Rust makes a very specific promise: **performance comparable to C and C++, but with memory safety guaranteed at compile time.** No garbage collector. No runtime overhead. No null pointer exceptions. No data races.

That last point deserves emphasis. Most modern languages prevent some of these problems at runtime — by throwing exceptions or pausing to run the garbage

collector. Rust prevents them at *compile time*. If your program compiles, it cannot have a dangling pointer, cannot have a use-after-free bug, and cannot have a data race in safe code.

That's why Rust has been voted the **most-loved programming language on the Stack Overflow Developer Survey for nine consecutive years**. Developers who learn it don't want to go back.

And it's not just a niche curiosity — Rust is now used in the **Linux kernel**, at **Microsoft** for Windows components, at **Google** for parts of Android, at **Amazon** for AWS infrastructure, and at **Mozilla** where it was created.

SLIDE 4 — The Curriculum

[Walk down the timeline]

I didn't just read blog posts. I followed a **formal 15-week self-study plan** with structured deliverables and progress reports.

My primary resources were *The Rust Programming Language* — affectionately called "the Book" by the community — and Google's *Comprehensive Rust* course, which is the curriculum Google uses internally to onboard engineers to Rust. I also have a copy of the comprehensive Rust PDF right here in the repository.

Weeks one and two were setup and fundamentals — getting cargo working, writing hello world, understanding types and variables.

Weeks three and four were the hardest: **ownership and borrowing**. This is the core of Rust, and it took real time to internalise.

From there I progressed through enums and pattern matching, traits and generics, error handling, iterators, concurrency, and finally async/await and the full web stack.

I filed **four formal progress reports** documenting what I learned each period — they're all in this repository. That accountability structure made a real difference.

SLIDE 5 — Core Concepts Mastered

Let me quickly walk through the core concepts, because these will come up in every project.

Ownership and Borrowing — Rust's most famous innovation. Every value has exactly one owner. You can have many shared references *or* one mutable reference, but never both at once. The compiler enforces this and it eliminates an entire class of bugs.

Enums and Pattern Matching — Rust enums aren't just labels — they carry data. And `match` forces you to handle every possible case. This is how you model game states, HTTP responses, and errors.

Traits and Generics — Rust achieves polymorphism through traits rather than inheritance. Generic functions are monomorphised at compile time — zero runtime overhead.

Error Handling — There are no exceptions in Rust. Functions return `Result<T, E>`. The `?` operator propagates errors cleanly up the call stack. This makes error paths explicit and impossible to accidentally ignore.

Iterators and Closures — Rust's iterator chains are lazy and zero-cost. `map`, `filter`, `fold` — these compile down to the same machine code as a hand-written loop.

Async/Await — The Tokio runtime enables thousands of concurrent connections on a small thread pool. This is how all three web projects handle HTTP traffic efficiently.

Workspace Crates — How to organise large projects with multiple modules as separate crates. Used in the RPG.

Web Stack — Axum routing, Tera and Askama templating, Serde serialisation, JWT authentication.

SLIDE 6 — Project 1: Foundational Exercises

[Point to each exercise]

The training repository begins with **foundational exercises** — one program per concept chapter in the Book.

`hello_world` was Day 1. `cargo run`, your first `println!` macro. Simple, but it's the ritual every Rust developer performs.

The exercises escalate: variables teaches shadowing and immutability. Functions introduces the distinction between statements and expressions — important in Rust. Structs shows how to define custom types and add methods.

The **guessing game** is the first complete program — it reads user input, generates a random number using the `rand` crate, and loops until the player guesses correctly. It exercises ownership, error handling, and `match` on `Ordering` all at once.

Then I went further and added a **GUI version** of the guessing game using `eframe` and `egui` — the same immediate-mode GUI framework used later in the Iron Age RPG. That was a significant milestone: going from a terminal program to a graphical application in Rust.

SLIDE 7 — Project 2: Ultra Game Suite

[Go through games enthusiastically]

The Ultra Game Suite is ten complete games shipped in a single Rust binary.

When you run it with no arguments, it launches a graphical menu built with eframe. You pick a game, play it, and return to the menu. Pass `--cli` for a pure terminal experience, or `--game 5` to jump straight to Checkers.

Each game lives in its own module under `src/`. Let me highlight a few:

Minesweeper — Full TUI board using `crossterm` for cursor control. Click cells, flag mines, win or lose.

Chess — Complete move validation including check and checkmate detection.

Tic-Tac-Toe — A minimax AI that plays perfectly. You literally cannot win; the best you can do is draw.

Blackjack — This one has a custom ASCII card-flip animation. When a card is dealt, it animates expanding from a sliver to full width, then flips to reveal the face — all achieved with crossterm cursor manipulation. It's one of the most technically creative pieces of code in the repo.

Poker — Five-card draw with hand evaluation: pair, two pair, straight, flush, full house, all the way to royal flush.

The key Rust concepts here are **trait objects** for the pluggable game architecture, **enums** for game state machines, and **crossterm** for terminal control.

SLIDE 8 — Project 3: Iron Age RPG

[Highlight the workspace architecture]

Iron Age RPG is the **flagship project** in the training repo, and it's the one that best demonstrates professional software engineering in Rust.

It uses a **Cargo workspace** — ten crates, each with a single responsibility:

- `core` — shared types and error definitions
- `character` — player and NPC stats, levelling, experience
- `combat` — the turn-based combat engine
- `inventory` — items, equipment, capacity
- `world` — the map, areas, sub-areas, enemies, bosses
- `narrative` — quest engine, 36 side quests, story arcs
- `crafting` — recipe system, resource gathering
- `data` — JSON and TOML asset loading via Serde

- `game` — the CLI game loop *and* the egui GUI
- `minesweeper` — a standalone minesweeper binary included in the workspace

The game ships as two binaries: `cargo run --bin iron-age-rpg` for the terminal, and `cargo run --bin iron-age-rpg-gui` for the graphical version.

There are **84 automated tests** across the workspace. This is real software engineering discipline — not just writing code, but verifying it behaves correctly.

The world has five sub-areas with boss encounters, twelve enemy types, and over three dozen side quests. The data is entirely driven by JSON and TOML files — adding new content doesn't require touching Rust code.

SLIDE 9 — Project 4: Anomalous Inquiry

[Emphasise the no-JavaScript philosophy]

Anomalous Inquiry is the first of three **fully-deployed web applications** in the portfolio.

It's a neutral, documentary-style research platform for investigating anomalous experiences — UAP incidents, parapsychology research, near-death experiences, remote viewing, and so on.

The entire application is written in Rust. The tech stack is: - **Axum 0.7** for the HTTP layer - **Tera** for server-side HTML templates - **pulldown-cmark** for rendering Markdown articles - **Tokio** for the async runtime - **printpdf** for per-article PDF export - **rss crate** for an auto-generated RSS feed

And here's the philosophy: **zero client-side JavaScript.** The whole application renders server-side. It works on any device, including text-based browsers. This is a deliberate design choice that demonstrates Rust's capability to handle the full web stack without reaching for a JavaScript framework.

The admin panel is protected by cookie authentication. The comment system is moderated. There's a CE1–CE5 close encounter archive with an interactive timeline — all rendered server-side.

It's deployed on **Render.com** using a `render.yaml` configuration file for infrastructure-as-code. Build time on the free tier is about three to six minutes. The whole thing compiles to a single self-contained binary.

SLIDE 10 — Project 5: Esoteric Wisdom

[This is the most architecturally sophisticated — convey that]

Esoteric Wisdom is the most architecturally advanced application in the portfolio. It's live right now at **esoteric-wisdom.onrender.com**.

It's a comprehensive spiritual portal with over **140 content pages** spanning every major esoteric tradition — Hermeticism, Kabbalah, Tantra, Sufism, Druidism, you name it. It has a full tarot card reader with **15 historic decks**, a personal journal with mood tracking, and **user authentication**.

Let me talk about the tech choices because they're significant:

Askama templates compile all 164 HTML templates **at build time**. Type errors in templates are caught by the Rust compiler. This is unique to the Rust ecosystem — you literally cannot deploy a template with a broken variable reference because it won't compile.

Argon2 for password hashing — the winner of the Password Hashing Competition and considered the gold standard today.

JWT tokens stored in HTTP-only cookies for stateless authentication. No server-side session store required. The application is horizontally scalable by design.

Arc<RwLock<AppState» is the idiomatic Rust pattern for sharing mutable state across concurrent async request handlers. The journal entries, user list, and tarot deck data all live in this shared structure, and the borrow checker guarantees no data races.

The aesthetic is deliberately immersive — animated star fields, aurora effects, sacred geometry — all done with Tailwind CSS, no JavaScript frameworks.

SLIDE 11 — Project 6: GeoPolSim

[Explain the vision even if the repo is still developing]

GeoPolSim is a **geopolitical simulation engine** — a Rust application that models nation-state dynamics, economic interactions, and geopolitical events.

Why is Rust particularly well-suited for simulation? Several reasons:

Deterministic execution. No garbage collector means no pauses mid-tick. Simulations need reproducible, predictable timing — Rust delivers that in a way that Python or Java cannot guarantee.

Data modelling. Rust's structs and enums map naturally to simulation entities: a `Nation` struct owns its `Resources`, `Alliances`, and `Territory`. The ownership model literally mirrors the conceptual model.

Performance. Hundreds or thousands of simulated entities can be processed per tick without latency spikes. If you need parallelism, **Rayon** makes it trivial

to parallelise iteration over entities, and the borrow checker guarantees those parallel iterations cannot race.

Serde. Adding `#[derive(Serialize, Deserialize)]` to a struct gives you automatic save/load of simulation state as JSON. One attribute, zero boilerplate.

Extensibility. A trait-based plugin architecture lets you add new simulation modules that implement a `Simulatable` trait and cleanly integrate into the main engine loop without modifying existing code.

SLIDE 12 — Common Patterns Across All Projects

Here's something I didn't expect when I started: the same Rust idioms appear in **every single project**, from the tiny guessing game to the full-stack web application.

`Result<T, E>` and the `?` operator — Used everywhere. The moment a function can fail, it returns `Result`, and `?` propagates the error up the stack. No exceptions, no ignored errors, no silent failures.

`#[derive(Serialize, Deserialize)]` — One attribute gives a struct full JSON and TOML read/write capability. RPG data assets, web journal entries, article metadata, simulation state — all use this.

`Arc<Mutex<>>` and `Arc<RwLock<>>` — Thread-safe shared state. Appears in every concurrent context: the web servers, the game state manager, anywhere multiple tasks share data.

Trait objects — The Ultra Game Suite uses `dyn Game` so the GUI launcher holds any game variant behind a single pointer. Polymorphism without class hierarchies.

Cargo workspaces — Iron Age RPG's ten-crate workspace is the same structure used in real enterprise Rust projects. Compile times stay fast, module boundaries stay clean.

`async fn` and `.await` — All three web projects. Tokio schedules thousands of concurrent HTTP connections on a small thread pool. The code looks synchronous; the runtime makes it concurrent.

SLIDE 13 — Progress at a Glance

[Pause. Let the numbers land.]

Let me put this in perspective.

Six shipped projects. Ten complete games. Ten workspace crates. Eighty-four automated tests. A hundred and forty-plus content pages. A hundred and sixty-four compiled templates. Thirty-six side quests. Fifteen tarot decks.

This is not tutorial code. This is a real body of work.

SLIDE 14 — What I Really Learned

[Be personal and genuine here]

The technical skills are one thing. But here's what I *really* learned.

The compiler is your best teacher. Every borrow check error is a lesson. Rust's error messages are famously good — they don't just tell you what's wrong, they explain *why* and often suggest the fix. After fighting the compiler enough times, you start to understand the principles behind its decisions.

Build, don't just read. I could have read the Book twice and still not understood ownership at a deep level. It only became real when I built something that broke — and debugged it. Every game, every route handler, every failing test burned a concept into memory that reading alone couldn't.

Rust scales from hello_world to production. The exact same idioms — `Result`, `match`, `impl Trait` — appear in the one-file guessing game and the deployed web server. There's no "advanced Rust" that's secretly a different language. It's the same language all the way down.

Fearless refactoring. Once the borrow checker is satisfied, large structural changes feel safe. The compiler catches broken invariants that would cause runtime errors in a dynamically typed language. I refactored the RPG's world module significantly and trusted the compiler to catch everything.

The ecosystem is mature. Tokio, Axum, Serde, Askama, eframe, crossterm — these are world-class crates with excellent documentation, active maintenance, and real production use. You are never alone as a Rust developer.

SLIDE 15 — What's Next

I'm not finished. Here's what I'm working toward:

Advanced lifetimes and GATs — Generic Associated Types and Higher-Ranked Trait Bounds are the frontier of Rust's type system. Understanding these will unlock patterns that aren't possible in any other mainstream language.

Database integration — `sqlx` or `SeaORM` for async PostgreSQL. Right now the web projects store data in memory; adding a database makes them production-ready.

WebAssembly — Compile Rust directly to Wasm and run it in the browser at near-native speed. This is where Rust's web story becomes genuinely exciting.

Embedded systems — `no_std` Rust on microcontrollers. No operating system, no allocator, just Rust talking directly to hardware. This is where the zero-overhead guarantee matters most.

Distributed systems — gRPC with `tonic`, message queues, eventually a distributed version of GeoPolSim that runs across multiple nodes.

Open source contribution — Contributing to the crates I've used. That's the mark of real competence.

SLIDE 16 — Links & Thank You

[Leave this slide up during Q&A]

Thank you.

All four repositories are public on GitHub. The live Esoteric Wisdom portal is available right now. Links are on the screen.

I'm happy to walk through any part of the code in more detail — the RPG workspace architecture, the Blackjack animation, the JWT authentication flow, anything you're curious about.

Any questions?

TIMING GUIDE

Section	Slides	Target Time
Opening + Why Rust	1–3	~2 min
Curriculum + Concepts	4–5	~2.5 min
Foundational Exercises	6	~1.5 min
Ultra Game Suite	7	~2 min
Iron Age RPG	8	~2 min
Anomalous Inquiry	9	~1.5 min
Esoteric Wisdom	10	~1.5 min
GeoPolSim	11	~1 min
Cross-Project Patterns	12	~1.5 min
Metrics + Lessons + Next	13–15	~2 min
Links + Q&A	16	~2 min
Total		**~19 min**

End of script